Karthik Uppuluri Electrical and Computer Engineering

ELEC 547 Assignment # 2

Seam Carving

Summary of Seam Carving for Content-Aware Image Resizing (Shai Avidan and Ariel Shamir[1])

Objective of the paper: Content-aware resizing (reduction and expansion) of Images

Idea proposed: Successive reduction or insertion of seams reduces or enlarges the size of the image, where a seam is defined as a low-energy 8-connected path of pixels on an image from top to bottom or left to right.

Figure.1 Algorithm

Energy functions algorithm can support:

Proposed algorithm supports different energy functions like gradient magnitude, entropy, visual saliency and eye gaze movement. A scribble based interactive user interface has also been provided to add weights to the energy function.

Effective energy functions:

Gradient magnitude and Histogram of gradients are proved to be working well in most of the cases.

Major ideas discussed:

a. *Column Insertion/Removal*: If the input image is of size n x m and the output image required is of size n x m' and m' > m, this can be simply achieved by inserting (m'-m= c) c vertical seams to the input image, else if m' < m, c (m-m') vertical seams re removed from the input image. Same aspect ratio correction can be achieved by increasing the number of rows (inserting horizontal seams)by a factor of m/m'
b. *Image Retargeting*: If the input image is of size n x m and the output image is of size n' x m', then the optimal order in which seams are removed or inserted is a problem of optimizing the following objective function

$$\min_{s^x, s^y, \alpha} \sum_{i=1}^{k} E(\alpha_i s_i^{\,x} + (1-\alpha_i) s_i^{\,y})$$

Where s^x *and* s^y are vertical and horizontal seams; k = r +c where r = (m-m'), c = (n-n'); α_i is used to determine whether in a step (i) we remove vertical or a horizontal column. α_i is {0, 1}. A transport map T(r,c) is created which gives the minimal cost required to obtain an image of size (n- r) x (m-c).Transport map is computed using dynamic programming, T(0,0) =0 and remaining entries are filled by choosing either to remove a horizontal seam from an image or a vertical seam. To achieve a target size if n' x m' where n' = n-r, m'=m-c we just back track from T(r,c) to T(0,0) and apply removal operations.

c. *Image Enlarging*: Each optimal vertical seam 's' is computed on the image and the pixels of 's' are duplicated by averaging left and right neighbors(top and bottom in the horizontal case). A stretching artifact is likely to be created if we repeat the above processes. Therefore to increase the size by 'C' first C seams for removal are found and those seams are duplicated to achieve the enlarged Image.
d. *Content Amplification*: Content amplification can be achieved by first scaling the image and applying seam carving to obtain its original size.
e. *Object Removal*: A user can mark the objects which should be removed from the image. And seam removal is applied till all the pixels of the object marked are removed, followed by seam insertion to regain its original size.
f. *Multi*-size Images: Multi-size images are one way useful in speeding up the computation and in other way satisfying the need for real time image resizing. An entire range of retargeting sizes from 1 x1 to m x n and even more to N' X M' where N' > n and M' >m are computed as a preprocessing step. Both vertical and horizontal index maps are computed which maps the co-ordinates of the pixels to the seam number in which those pixels are removed. So these index maps can be readily used to insert or delete seams both horizontally and vertically in real-time.

Limitations of the algorithm: Seam carving fails where the image has no or very few 'less important areas' and not all the times the results are automatic.

Dynamic Programming algorithm:

Dynamic programming is a method of solving a complex problem by breaking it into simpler sub-problems. It's especially efficient in the case of a problem having overlapping sub problems, (example Fibonacci sequence) and optimal substructure (optimality achieved in individual sub-problems leads to optimal solution of the entire problem).Dynamic programming stores the results of sub-problems and thereby reduces the redundancy in the calculation which decreases the time to solve a problem. Example, if we were to calculate the value of F(5) in Fibonacci series we will calculate the value of F(2) 3 times [2], as the dynamic programming stores the results of sub-problems the time complexity of the problem reduces. Similarly solving for a minimum cost seam over the entire image is a problem with many redundant calculations and dynamic programming approach would be a better and time efficient way to solve.

Benefits:

1. Faster way to solve a particular set of problems like mentioned above.

2. Optimal solution is guaranteed unlike greedy algorithms where the algorithms are faster but the solution might not be optimal.

Experimentation with Seam carving

Method

- **Energy map** of an Image is calculated. (Sum of gradients in each channel)
- A **cost matrix** is then calculated where the cost of pixels in the first row is same as the cost of the energy function value at respective positions. For all other pixels the cost of reaching the pixel is sum of the energy function value at that pixel added to the minimum of energy function value of three nearest top neighbors

  $$C(i,j) = e(i,j) \ if \ \ i == 1$$

  $$C(i,j) = e(i,j) + \min(C(i-1,j-1), C(i-1,j), C(i-1,j+1)) \ \ (for \ all \ i \neq 1)$$
- After finding the cost matrix, a **minimum energy seam** is found from bottom to top using dynamic programming. First minimum value in the bottom row of the cost matrix is found and is tracked up to the top edge and all the co-ordinates of the seam are recorded.
- Recorded co-ordinates of the minimum energy seam are then used to remove the seam from the image. And steps a, b, c and d are repeated until all the required number of seams are removed.
- For seam insertion the order in which seams are removed and their respective co-ordinates are recorded. Pixel values of the seam are duplicated using the pixel value of the corresponding co-ordinates in the original image. And the seams are added to the image in the order of their removal.

Test results of seam insertion (3 images) and seam removal (4 images) are provided below.

Problems and surprises: Seam insertion didn't give proper results if my duplicating pixel value is average of its left and right neighbors

Analysis of test results:

1. Seam removal worked well in the images where there are more "less important" areas (fig.1, 4, 6)
2. Seam insertion gave a stretching effect if the number of columns inserted is greater than or equal to 50 % of original number of columns present (Seam insertion comparison Fig.8)

Results

Figure1. Input Image (700 x 466)

Figure1.1 My result after removing 350 columns

Figure1.2 Result from the paper (350 x 466)

Figure2. Input Image (500 x 375)

Figure2.1 My result after removing 200 columns

Figure2.2 Result from the paper (300 x 375)

Figure3. Input Image (486 x 738)

Figure3.2 My result after inserting 243 columns

Figure3.3 Result from the paper (729 x 738)

Other Results

Figure4. Input Image (800 x 278)

Figure4.1 My result after removing 200 columns (600 x 278)

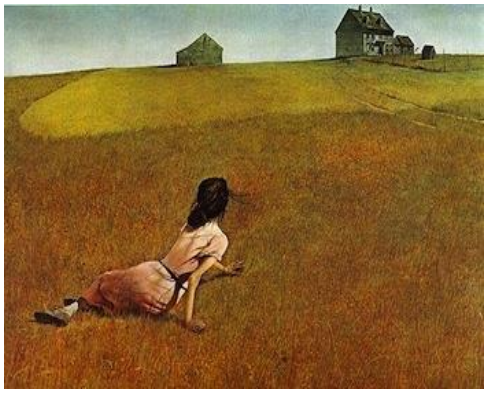

Figure5. Input Image (350 x 248)

Figure5.1 My result after inserting 150 columns (500 x 248)

Figure6. Input Image (400 x 300)

Figure6.1 My result after removing 100 columns (300 x 300)

Figure7. Input Image (500 x 375)

Figure7.1 My result after inserting 153 columns

Seam insertion comparison (Stretching effect is evident in images where number of columns inserted is more than 50 % of original number of columns present)

Figure.8 Less than 50% of original number of columns inserted

Figure.8.1 50% of the original number of columns inserted

References

1. http://www.faculty.idc.ac.il/arik/SCWeb/imret/
2. http://www.webmath.iitkgp.ernet.in/breadth/ma23011/lectnotes07/Dynamic_Programming-4.pdf
3. Images used from http://cs.brown.edu/courses/cs129/results/proj3/taox/